

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Rotaru Anastasia
Group FAF-242

Report

Laboratory Work No. 6
Operating Systems
Topic: Inside the xv6 File System

Checked by:
Reițman Patricia, asist. univ.

Chisinau – 2026

Contents

1 INTRODUCTION AND OBJECTIVES **3**

1.1 Context of the Work 3

1.2 Main Objectives 3

2 EXPLORING INTERNAL ORGANIZATION IN XV6 **4**

2.1 Execution Captures and Results 4

2.2 Data Interpretation 4

3 THE MYSTAT PROGRAM AND HARD LINK MECHANISMS **6**

3.1 Implementation of `user/mystat.c` 6

3.2 Testing Session and Link Tracking 6

3.3 Observations and Partial Conclusions 7

4 EVALUATING FILE SYSTEM LIMITS **8**

4.1 Inspecting Architectural Parameters 8

4.2 Calculations and Detailed Analysis 8

5 IMPLEMENTING RECURSIVE SEARCH (FIND) **9**

5.1 Source Code for `user/find.c` 9

5.2 Verification of Functionality 10

5.3 Stretch Goal Evaluation 10

CONCLUSIONS **11**

1 INTRODUCTION AND OBJECTIVES

1.1 Context of the Work

The xv6 operating system provides a compact and intuitive implementation of a UNIX-like file system[cite: 1]. Working within my development environment configured on the dedicated virtual machine on `Laptop-Rotaru`, analyzing how disk blocks and control structures are managed offers a clear perspective on low-level storage mechanisms.

1.2 Main Objectives

This laboratory work focuses on the following fundamental aspects:

- examining the structure of the root directory and how xv6 classifies file types;
- designing and implementing the custom utility `mystat` to query the `struct stat` structure;
- tracking the evolution of reference counters during hard link operations and file deletion;
- determining architectural storage limits by interpreting constants in the `kernel/fs.h` header file;
- developing the recursive `find` program designed to traverse directory hierarchies.

2 EXPLORING INTERNAL ORGANIZATION IN XV6

2.1 Execution Captures and Results

Starting the xv6 instance via QEMU from the laboratory environment, the contents of the main directory and a newly created directory `a` were queried using the `ls` command:

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
echo           2 4 35592
forktest       2 5 18136
grep           2 6 44184
init           2 7 35976
kill           2 8 35512
ln             2 9 35304
ls             2 10 42880
mkdir          2 11 35568
rm             2 12 35552
sh             2 13 58312
stressfs       2 14 36416
usertests      2 15 206800
grind          2 16 51824
wc             2 17 37728
zombie         2 18 34848
logstress      2 19 37560
forphan        2 20 36352
dorphan        2 21 35800
sync           2 22 34944
sleep          2 23 35280
args           2 24 35272
console        3 25 0

$ mkdir a
$ ls a
.              1 26 32
..             1 1 1024
```

2.2 Data Interpretation

a) **Meaning of types 1, 2, and 3:** Consulting the `kernel/stat.h` file, the numerical codes correspond to the following entities:

- `T_DIR (1)`: Directory.
- `T_FILE (2)`: Regular data file.
- `T_DEVICE (3)`: Special hardware device (such as the system console).

b) **File system root and circular references:** The root directory always holds the inode numbered **1** (defined by the constant `ROOTINO`). Both the current reference (`.`) and the parent reference (`..`) point to the exact same inode **1**, because the root directory does not feature an upper directory in the hierarchy.

c) **Size of empty directories:** The empty directory `a` occupies a size of **32 bytes**. Since each directory entry (`struct dirent`) has a fixed size of 16 bytes (2 bytes for the inode number `inum` and 14 bytes for the string name `name`), exactly two entries result (32/16 = 2), represented by `.` and `..`.

3 THE MYSTAT PROGRAM AND HARD LINK MECHANISMS

3.1 Implementation of `user/mystat.c`

The utility was written to retrieve metadata through the `stat()` system call:

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *typename(int t)
{
  switch (t) {
  case T_DIR:    return "directory";
  case T_FILE:   return "file";
  case T_DEVICE: return "device";
  default:       return "?";
  }
}

int main(int argc, char *argv[])
{
  struct stat st;

  if (argc < 2) {
    fprintf(2, "usage: mystat <file...>\n");
    exit(1);
  }

  for (int i = 1; i < argc; i++) {
    if (stat(argv[i], &st) < 0) {
      fprintf(2, "mystat: cannot stat %s\n", argv[i]);
      continue;
    }
    printf("%s: %s, inode %d, links %d, size %d bytes\n",
           argv[i], typename(st.type), st.ino, st.nlink, (int)st.size);
  }
  exit(0);
}
```

3.2 Testing Session and Link Tracking

```
$ mystat README . console
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 1, size 1024 bytes
console: device, inode 27, links 1, size 0 bytes

$ ln README r2
$ mystat README r2
README: file, inode 2, links 2, size 2441 bytes
r2: file, inode 2, links 2, size 2441 bytes

$ rm README
$ mystat r2
r2: file, inode 2, links 1, size 2441 bytes

$ cat r2 | wc
48 336 2441
```

3.3 Observations and Partial Conclusions

a) **Link counter dynamics:** Creating an alternative link via `ln README r2` causes both to share the same inode (**2**), with the value of `nlink` increasing from 1 to 2. When the original file `README` is removed via the `rm` command, its entry disappears from the directory table, and the link counter returns to 1. The actual content remains unaltered on disk, and the `cat r2 | wc` utility successfully reads all 2441 bytes.

b) **File deletion and block deallocation:** In the kernel implementation found in `kernel/sysfile.c`, the `sys_unlink()` function only decrements `ip->nlink` and updates the state on disk. Effective deallocation of data blocks does not occur at this point; instead, it is handled in `kernel/fs.h` / `kernel/fs.c` through the `iput()` function when the in-memory reference count drops to zero and no active links remain associated with the inode.

c) **Structure of an inode in xv6 (`struct dinode`):** Source code analysis reveals a simplified structure that omits complex elements such as POSIX permissions or UIDs, strictly containing: file type, major/minor device numbers, active link count, logical size, and the address vector for direct and indirect blocks.

4 EVALUATING FILE SYSTEM LIMITS

4.1 Inspecting Architectural Parameters

Identifying constants through dedicated commands on the kernel sources:

```
$ grep -n "define␣BSIZE\|define␣NDIRECT\|define␣NINDIRECT\|define␣DIRSIZ" kernel/fs.h
5:#define BSIZE   1024 // block size
26:#define NDIRECT   12
27:#define NINDIRECT (BSIZE / sizeof(uint))
54:#define DIRSIZ 14

$ make clean ; make fs.img 2>&1 | grep nmeta
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
```

4.2 Calculations and Detailed Analysis

a) **Directory name length restriction:** Testing the creation of a directory with an extended name (`averyveryverylongname`) shows that it gets truncated and displayed as `averyveryveryl`. This behavior stems directly from the `DIRSIZ` macro value fixed at **14 characters**.

b) **Determining maximum file size:** The maximum capacity supported for a file in xv6 is calculated based on direct blocks and those allocated through the indirect block:

$$\text{Block Size } (BSIZE) = 1024 \text{ bytes} \tag{4.1}$$

$$\text{Pointer Size} = \text{sizeof}(\texttt{uint}) = 4 \text{ bytes} \tag{4.2}$$

$$\text{Number of Indirect Entries } (NINDIRECT) = \frac{1024}{4} = 256 \text{ blocks} \tag{4.3}$$

$$\text{Total Addressable Blocks} = NDIRECT + NINDIRECT = 12 + 256 = 268 \text{ blocks} \tag{4.4}$$

$$\text{Maximum File Capacity} = 268 \times 1024 = 274,432 \text{ bytes } (\approx 268 \text{ KB}) \tag{4.5}$$

c) **Disk block distribution:** According to the data reported by the `nmeta` line, the disk image contains a total of **2000 blocks**, divided into:

- 47 metadata blocks (1 boot block, 1 superblock, 31 logging/log blocks, 13 inode blocks, and 1 bitmap block).
- 1953 blocks exclusively allocated for storing useful data for files and directories.

5 IMPLEMENTING RECURSIVE SEARCH (FIND)

5.1 Source Code for `user/find.c`

The implemented recursive search module traverses `dirent` entries, ignores empty slots and parent/current entries (`/ ..`), and optionally integrates the extended `-n` flag:

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"
#include "kernel/fcntl.h"

void find(char *path, char *target, int show_ino)
{
  char buf[512];
  char *p;
  int fd;
  struct dirent de;
  struct stat st;

  if ((fd = open(path, O_RDONLY)) < 0) {
    fprintf(2, "find: cannot open %s\n", path);
    return;
  }
  if (fstat(fd, &st) < 0 || st.type != T_DIR) {
    close(fd);
    return;
  }
  if (strlen(path) + 1 + DIRSIZ + 1 > sizeof(buf)) {
    fprintf(2, "find: path too long\n");
    close(fd);
    return;
  }

  strcpy(buf, path);
  p = buf + strlen(buf);
  *p++ = '/';

  while (read(fd, &de, sizeof(de)) == sizeof(de)) {
    if (de.inum == 0) continue;
    if (strcmp(de.name, ".") == 0 || strcmp(de.name, "..") == 0) continue;

    memmove(p, de.name, DIRSIZ);
    p[DIRSIZ] = 0;

    if (stat(buf, &st) < 0) continue;

    if (strcmp(de.name, target) == 0) {
      if (show_ino)
        printf("%s %d\n", buf, st.ino);
      else
        printf("%s\n", buf);
    }

    if (st.type == T_DIR)
      find(buf, target, show_ino);
  }
  close(fd);
}

int main(int argc, char *argv[])
```

```
{
  int show_ino = 0;
  if (argc < 3) {
    fprintf(2, "usage: find <path> <name> [-n]\n");
    exit(1);
  }
  if (argc >= 4 && strcmp(argv[3], "-n") == 0)
    show_ino = 1;
  find(argv[1], argv[2], show_ino);
  exit(0);
}
```

5.2 Verification of Functionality

The test hierarchy generated on the file system image was validated through the following commands:

```
$ mkdir a
$ mkdir a/b
$ echo x > a/b/r2
$ echo y > a/r2
$ ln cat r2
$ find . r2
./r2
./a/b/r2
./a/r2

$ find a b
a/b

$ find . r2 -n
./r2 2
./a/b/r2 31
./a/r2 32
```

5.3 Stretch Goal Evaluation

Executing the command with the `-n` flag confirms that the `./r2` element shares the same inode (**2**) with the original `cat` utility, whereas files subsequently created under nested directories (`./a/r2` and `./a/b/r2`) hold distinct unique inodes (32 and 31 respectively), verifying the correctness of the hard link referencing mechanism.

CONCLUSIONS

This laboratory work facilitated a practical understanding of how the file system is designed within xv6. By analyzing the structures in `kernel/fs.h` and developing dedicated user-space utilities, essential concepts such as metadata-data separation, block management via inodes, and safe recursive directory traversal were highlighted.

Key aspects assimilated include:

- **Inode-Directory decoupling:** File names exist exclusively as text records in directory tables, mapped to a numerical inode identifier.
- **Effect of links:** Creating a hard link only updates reference counters, while actual space reclamation is deferred until active references are fully exhausted.
- **Architectural constraints:** Fixed file name lengths and direct/indirect addressing limits shape the scaling constraints of the system.

The performed study considerably reinforced systems programming competencies and deep comprehension of core-level operational workflows.